

Course Matcher Project Report

1. Executive Summary

Course Matcher is an AI-powered web application that takes a user's resume as input and recommends relevant courses based on semantic similarity. The app also shows similar profile matches and extracts recognizable skills from the uploaded resume. The current production-facing version uses Flask for the backend, vanilla JavaScript for browser interactions, and a custom HTML/CSS frontend.

The core idea is simple:

- Read the resume text from a PDF or TXT file.
- Convert that text into an embedding vector using a sentence transformer model.
- Search FAISS indexes for similar resumes and relevant courses.
- Re-rank course candidates using MMR so results are not too repetitive.
- Return structured JSON to the frontend and render the results in a clean UI.

2. Problem Statement

People often have resumes but do not know which courses are the best next step for their skill level and background. Keyword search is too shallow because resumes and course descriptions may express the same idea in different words. This project solves that by using semantic search rather than plain keyword matching.

3. Business Value

- Helps users discover relevant upskilling opportunities faster.
- Makes recommendations feel more personalized than normal search.
- Shows both current skill signals and learning gaps.
- Demonstrates a practical AI workflow that combines NLP, search, ranking, and UX.

4. High-Level Architecture

User Flow

- User opens the web app.
- User uploads a PDF or TXT resume.
- Frontend sends the file to `/api/recommend`.
- Backend extracts text and generates embeddings.
- Backend searches vector indexes and ranks the results.
- Backend returns JSON with skills, profile matches, and courses.
- Frontend renders chips, cards, and links for the results.

Layers

- Presentation Layer: HTML, CSS, JavaScript
- API Layer: Flask
- AI/NLP Layer: SentenceTransformers embeddings
- Retrieval Layer: FAISS vector search
- Ranking Layer: MMR reranking with cosine similarity
- Data Layer: JSON, pickle, FAISS index files

5. Current Tech Stack

Backend

- Python
- Flask
- NumPy
- FAISS
- SentenceTransformers
- scikit-learn
- PyMuPDF

Frontend

- HTML5
- CSS3
- Vanilla JavaScript

Data / Artifacts

- `resume\_metadata.json`
- `courses\_metadata.pkl`
- `resume\_index.faiss`
- `courses\_index.faiss`
- `courses.json`

6. Easy Explanation Of Each Technology

Python

Python is the main programming language of the app. It is used because it has strong libraries for AI, NLP, backend APIs, and data processing.

Flask

Flask is a lightweight web framework. It helps the app expose URLs like ``` and `/api/recommend``. You can think of it as the layer that receives browser requests and sends responses back.

SentenceTransformers

SentenceTransformers converts text into embeddings. An embedding is a list of numbers that represents meaning. Similar texts produce vectors that are close together in vector space.

FAISS

FAISS is a vector similarity search library. It stores embeddings and quickly finds which existing items are closest to a new query embedding. That is what makes semantic search fast.

NumPy

NumPy handles arrays and numeric operations. Embeddings are stored and processed as NumPy arrays before being searched or ranked.

scikit-learn

scikit-learn is used here for cosine similarity. Cosine similarity measures how aligned two vectors are and helps compare the semantic closeness of embeddings.

PyMuPDF

PyMuPDF reads text from uploaded PDF resumes. Without it, the backend would not be able to convert a resume PDF into machine-readable text.

HTML

HTML defines the structure of the frontend page: sections, buttons, cards, text blocks, upload input, and placeholders for results.

CSS

CSS controls the visual design of the app: layout, colors, fonts, spacing, responsiveness, and the polished card-based interface.

JavaScript

JavaScript handles frontend interactivity. It listens for the file upload form submit, sends the request to the backend, and renders the returned results dynamically.

Pickle

Pickle is a Python serialization format. In this app it stores course metadata after preprocessing so the backend can load course details efficiently.

JSON

JSON is a lightweight data format. It is used for resume metadata storage and API responses.

7. Main Files And Their Roles

``webapp.py``

This is the main backend application.

Responsibilities:

- Initializes the Flask app
- Loads the embedding model
- Loads the FAISS indexes and metadata
- Extracts text from uploaded files
- Extracts skills from resume text
- Searches similar profiles and courses
- Re-ranks courses using MMR
- Exposes the API endpoints

`templates/index.html`

This is the main page structure.

Responsibilities:

- Defines the upload section
- Defines containers for skills, profile matches, and course recommendations
- Loads CSS and JavaScript assets

`static/styles.css`

This file contains the custom visual design.

Responsibilities:

- Defines colors and theme variables
- Builds responsive layout
- Styles cards, chips, upload area, status pill, and buttons

`static/app.js`

This file controls the browser-side behavior.

Responsibilities:

- Handles file selection UI
- Sends uploaded resumes to the backend
- Shows loading and error states
- Renders skills, profiles, and course results dynamically

`final2.py`

This is the preserved Streamlit-era prototype.

Responsibilities:

- Single-file recommendation prototype
- Demonstrates course recommendation flow

- Uses the same core AI concepts in an older UI

``resume/resume_mdata.py``

This script builds the resume FAISS index while keeping metadata aligned with vectors.

``scraper.py``

This script fetches course-like data and writes it to a CSV.

8. Functions In ``webapp.py``

``load_model()``

Loads the sentence transformer model. It is cached with ``lru_cache`` so the model is loaded once and reused.

``load_resume_data()``

Loads the resume FAISS index and resume metadata.

``load_course_data()``

Loads the course FAISS index and course metadata.

``extract_text_from_pdf(file_storage)``

Extracts text from an uploaded PDF file using PyMuPDF.

``extract_text(file_storage)``

Chooses how to read the uploaded file:

- If PDF: call the PDF extractor
- Else: treat it as UTF-8 text

``extract_skills_from_resume(resume_text, known_skills)``

Performs basic skill extraction by checking whether known skill keywords appear in the resume text.

``safe_faiss_search(index, query_embedding, top_k)``

Searches FAISS safely.

Why it matters:

- Clamps ``k`` so it never asks for more results than exist
- Filters out invalid ``-1`` ids
- Prevents crashes and bad lookups on small datasets

``mmr(query_embedding, candidate_embeddings, k=5, lambda_param=0.65)``

Applies Maximal Marginal Relevance.

What it does:

- Chooses results relevant to the query

- Penalizes near-duplicate candidates
- Produces more diverse recommendations

``build_response(resume_text)``

This is the core application pipeline.

What it does:

- Loads model and indexes
- Embeds the resume
- Extracts skills
- Finds similar resumes
- Finds candidate courses
- Embeds the candidate course descriptions
- Re-ranks them with MMR
- Returns the final response structure

``index()``

Serves the main HTML page at ``/``.

``health()``

Returns a simple health check response at ``/api/health``.

``recommend()``

Main API endpoint at ``/api/recommend``.

What it does:

- Validates file upload
- Extracts resume text
- Calls ``build_response``
- Returns JSON or error messages

9. APIs

``GET /``

Purpose:

- Serves the main frontend page

Response:

- HTML page

`GET /api/health`

Purpose:

- Quick health/status endpoint

Example response:

```
{ "status": "ok" }
```

`POST /api/recommend`

Purpose:

- Accept a resume file and return AI-generated recommendation results

Input:

- Multipart form-data
- Field name: `resume`

Output shape:

```
{ "skills": ["Python", "SQL"], "profiles": [ { "name": "Alice", "summary": "Software engineer skilled in Python, Django, and REST APIs." } ], "courses": [ { "title": "Course Name", "university": "Provider Name", "description": "Course description text", "url": "https://example.com/course", "skills": ["Python", "ML"] } ] }
```

Error cases:

- Missing file upload
- Unsupported or unreadable file
- Empty extracted text
- Missing FAISS or metadata files
- Runtime errors in recommendation flow

10. Data Flow

Input

- Resume uploaded as PDF or TXT

Processing

- Convert resume into text
- Convert text into embedding
- Search vector indexes
- Apply ranking logic
- Format final JSON

Output

- Extracted skills
- Similar profiles
- Recommended courses

11. Why This Is An AI App

This is an AI app because the app does not rely only on fixed rules or exact keyword matching. It uses a trained language representation model to understand text semantically.

That means:

- It can match related meanings, not just identical words
- It transforms unstructured text into embeddings
- It performs similarity search on those AI-generated vectors
- It ranks and personalizes results from the user's input

In short, the core decision-making layer is based on machine-learned text representations.

12. AI Elements Present In The App

1. Unstructured Text Understanding

The app accepts free-form resume text rather than rigid form fields.

2. Embedding Model

The sentence transformer model converts natural language into vector representations.

3. Semantic Search

FAISS retrieves items by vector similarity rather than strict keywords.

4. Recommendation Logic

The app recommends learning content based on semantic closeness to user context.

5. Ranking And Diversification

MMR balances relevance and diversity to improve recommendation quality.

6. Personalized Output

The response changes based on the actual resume content uploaded by the user.

13. Current Strengths

- Clear AI pipeline from upload to recommendation
- Uses semantic embeddings instead of basic search
- Fast retrieval with FAISS
- Clean frontend and API separation
- Safer handling for small datasets and invalid FAISS ids

- Simple enough to explain in interviews

14. Current Limitations

- Skill extraction is keyword-based rather than model-based
- No authentication or user accounts
- No persistent database
- No evaluation framework for recommendation quality
- Limited resume dataset
- No explanation scoring shown to users yet
- No async/background job model loading optimization

15. Future Improvements

- Add course match scores
- Add “why this course” explanations
- Improve skill extraction with NER or LLMs
- Add user history and saved recommendations
- Store data in a database
- Add admin ingestion pipeline for courses
- Add testing around API and ranking behavior
- Add batching, caching, and deployment configuration

16. Interview-Friendly Explanation Of The Core Pipeline

If someone asks, “How does your app work?”, a strong short answer is:

“Users upload a resume, the backend extracts the text, turns it into an embedding using SentenceTransformers, and then uses FAISS to find semantically similar resumes and courses. After that, I apply MMR reranking so the final course recommendations are both relevant and diverse. The backend exposes this through a Flask API, and the frontend renders the results dynamically with JavaScript.”

17. Interview Questions And Answers Prep

Architecture And Design

- What problem does this app solve?
- Why did you choose a web app architecture instead of a notebook-only prototype?
- Why did you separate the frontend from the backend?
- Why is Flask a good fit here?

- What would change if the app had to support thousands of users?
- How would you deploy this app in production?
- How would you store metadata in a more scalable way?
- How would you version models and indexes?
- What parts are CPU-heavy?
- What parts would you cache in production?

AI / ML / NLP

- What is an embedding?
- Why use SentenceTransformers here?
- What does `all-MiniLM-L6-v2` do?
- Why is semantic search better than keyword matching in this use case?
- How do you compare embeddings?
- What is cosine similarity?
- What is vector search?
- Why use FAISS?
- What is MMR and why did you use it?
- What are the limitations of embedding-based search?
- How would you evaluate recommendation quality?
- How would you reduce hallucination or poor-fit recommendations?
- How would you improve skill extraction beyond keyword matching?
- What happens if the resume contains unusual or unseen terms?
- How would you fine-tune the recommendation system?

Backend / API

- Walk me through the `/api/recommend` endpoint.
- How do you validate uploaded files?
- How do you handle empty or invalid input?
- Why did you add `safe\_faiss\_search`?
- Why does filtering invalid FAISS ids matter?
- Why do you use JSON responses?
- How would you add request logging?
- How would you add rate limiting?
- How would you secure file uploads?
- How would you write automated tests for the API?

Frontend

- Why did you move away from Streamlit?
- How does the frontend communicate with the backend?
- Why did you use vanilla JavaScript instead of React?
- How do you handle loading and error states?
- How does the UI stay responsive on mobile?
- How would you add progress indicators or animations?
- How would you improve accessibility?

Data And Indexing

- How were the FAISS indexes created?
- Why must metadata and index rows stay aligned?
- What happens if you request more FAISS neighbors than exist?
- What file formats are used and why?
- How would you update the course index when new courses arrive?
- How would you handle duplicate or stale course records?

Debugging And Reliability

- What bug did you fix in the reranking logic?
- Why did the MMR bug happen?
- Why was the `top\_k` FAISS usage problematic in the older code?
- How did you handle NumPy dependency compatibility?
- What edge cases did you defend against?
- What monitoring would you add in production?

Behavioral / Project Ownership

- What part of this project are you most proud of?
- What tradeoffs did you make to keep the scope manageable?
- What would you improve if you had one more week?
- How would you explain this project to a non-technical stakeholder?
- What did you learn from turning a prototype into a web app?
- If this became a startup product, what would your roadmap look like?

18. Sample Strong Interview Answers

Why is this an AI app?

"It is an AI app because the main recommendation logic depends on embeddings generated by a pretrained language model. Instead of exact keyword matches, the system understands semantic similarity between resumes and course descriptions, which makes the recommendations more intelligent and personalized."

Why use FAISS?

"FAISS is built for fast nearest-neighbor search over vectors. Since embeddings are high-dimensional vectors, FAISS makes retrieval much faster and more scalable than comparing every course against the query one by one."

Why use MMR?

"Raw similarity search often returns very similar courses. MMR helps balance relevance with diversity so the user sees a broader and more useful set of recommendations."

What is the most important engineering improvement you made?

"I improved reliability around search and ranking. I added safe FAISS querying to handle small indexes and invalid ids, and I fixed the MMR reranking logic so it does not crash when the candidate pool is smaller than the requested result count."

19. Resume Bullet Ideas Based On This Project

- Built an AI-powered resume-to-course recommendation web app using Flask, SentenceTransformers, FAISS, and custom frontend components.
- Implemented semantic search over resumes and course descriptions using embedding vectors and nearest-neighbor retrieval.
- Improved recommendation quality by adding MMR reranking for more diverse course suggestions.
- Designed and shipped a custom HTML/CSS/JavaScript frontend to replace a Streamlit prototype.
- Hardened the ranking pipeline with safer FAISS query handling, invalid-id filtering, and small-dataset edge case protection.

20. Final Takeaway

This project is a strong example of a practical AI application because it combines:

- NLP
- embeddings
- vector databases/search
- recommendation logic
- backend API design
- frontend product design

It is not just a model demo. It is an end-to-end AI product workflow with real input handling, ranking logic, structured responses, and a usable interface.